while iterating over the cursor, only one Eloquent model is retained in memory at any given time.

Internally, the **cursor** method implements the following features using PHP generators:

```
use App\Models\Office;
foreach (Office::where('location, 'NYC')->cursor() as $office)
{
 //Code Logic
}
```

An **Illuminate\Support\LazyCollection** instance is returned by the **cursor**. While only loading one model into memory at a time, lazy collections let you utilize many of the collection methods offered by standard Laravel collections:

```
use App\Models\User;
$users = User::cursor()->filter(function (User $user) {
 return $user->age > 18;
});
foreach ($users as $user) {
 echo $user->name;
}
```

Because it only keeps one Eloquent model in memory at a time, the **cursor** technique consumes far less memory than a standard query, but it will ultimately run out of space. This is because PHP's PDO driver stores all raw query results in its internal buffer. Consider switching to the lazy technique if you're working with a lot of Eloquent records.

## Advanced Subqueries

We can select all of the **locations** and the name of the office at each location using a single query by leveraging the subquery functionality offered by the query builder's **select** and **addSelect** methods:

```
use App\Models\Location;
use App\Models\Office;
return Location::addSelect(['last_office' =>
Office::select('name')
 ->whereColumn('Location_id', 'locations.id')
```